

LLM-Augmented High-Level Route Planning for Autonomous UAV Navigation

Prachit Gupta

May 27, 2026

Abstract

This report presents an LLM-augmented high-level route-planning framework for autonomous UAV navigation on the ModalAI Starling 2 platform. The system uses perception-derived obstacle descriptions, structured prompting, sparse waypoint generation, waypoint interpolation, verification-based safety filtering, and PX4 trajectory execution. The LLM is not used as a real-time motion planner. Instead, it acts as a semantic mission planner that converts natural language objectives and scene context into sparse NED-frame waypoints. Classical refinement and verification modules then enforce geometric and kinematic constraints before the verified trajectory is latched by the low-level controller. The implementation is available in the project repository: [starling-testing-ws](#).

1 Introduction and Motivation

Autonomous navigation for unmanned aerial vehicles (UAVs) has traditionally relied on classical path-planning algorithms such as A*, RRT, RRT*, optimization-based planners, and reinforcement-learning policies. These approaches provide strong geometric reasoning, collision checks, and mathematically grounded trajectory generation. However, such methods are limited in semantic awareness, contextual understanding, and human-interpretable decision making. Traditional planners are largely blind to natural language instructions, environmental semantics, and human intent. They usually require explicitly defined waypoint coordinates, handcrafted cost functions, or large amounts of environment-specific training data.

Recent advances in large language models (LLMs) and vision-language models (VLMs) have demonstrated strong capabilities in semantic reasoning, contextual awareness, human-aligned decision making, and generalization across domains. Unlike conventional path-planning methods, LLMs are trained on large human-generated datasets and therefore encode useful high-level reasoning patterns. This creates an opportunity to bridge the semantic gap between human intent and robotic execution.

Instead of requiring expert UAV operators to provide precise waypoint coordinates or manually designed mission specifications, natural language instructions can be translated into actionable navigation objectives. For example, instead of specifying an exact spatial coordinate, a user may provide an instruction such as:

“Navigate to the chair near the whiteboard while avoiding nearby obstacles.”

An LLM can interpret the semantic structure of the instruction, infer contextual relationships between objects, and generate sparse waypoint sequences corresponding to the intended high-level route. This significantly reduces dependency on expert operators and enables more intuitive human-robot interaction.

This motivation is especially relevant in modern human-centric environments where robots are expected to coexist and collaborate with humans. Earlier robotics applications often focused on environments inaccessible to humans. Modern autonomous systems, including self-driving vehicles, collaborative drones, and assistive robots, increasingly operate in shared environments. Consequently, high-level mission planning must align closely with human intent and contextual reasoning rather than relying only on geometric optimization.

The central hypothesis explored in this work is that LLM-generated sparse waypoint trajectories, when augmented with refinement and verification tools, can serve as effective high-level route plans for UAV navigation. Rather than relying on LLMs for low-level control or real-time trajectory optimization, the framework leverages their strengths in semantic reasoning, contextual interpretation, and generalization while delegating geometric feasibility and collision checking to traditional modules.

2 Problem Statement

The objective of this project is to investigate whether large language models can be reliably used for high-level UAV route planning when augmented with appropriate perception, refinement, and verification pipelines.

The project explores the following research questions:

1. Can LLMs generate semantically meaningful sparse waypoint trajectories from natural language instructions and environmental context?
2. Can refinement and verification pipelines compensate for the poor spatial interpretability and numerical inaccuracies of LLM-generated outputs?
3. Can LLM-generated high-level plans be deployed successfully on real UAV hardware platforms?
4. To what extent can LLM-based planning provide human-aligned and semantically aware navigation behavior compared to traditional planning approaches?

3 Related Work

More details are provided in `short_survey.pdf`.

Classical path-planning methods such as A*, RRT, and optimization-based planners remain highly effective in environments where geometric structure and obstacle maps are explicitly known. These methods can provide mathematical guarantees regarding completeness, collision avoidance, and optimality. However, they fundamentally lack semantic awareness and contextual understanding. They do not directly reason about movable obstacles, human intent, or contextual object relationships.

Learning-based approaches such as reinforcement learning and imitation learning have also demonstrated strong performance in UAV navigation tasks. However, these methods generally require large amounts of environment-specific training data and often fail to generalize reliably to unseen environments.

Recent research involving foundation models, LLMs, and VLMs has demonstrated promising capabilities in robotics, navigation, reasoning, and multimodal perception. Prior work such as FM-Planner has explored the use of LLMs and VLMs for drone navigation by generating sparse waypoints using textual prompts and visual encoders. This project follows a similar conceptual

direction while focusing specifically on hardware deployability, high-level semantic route planning, verification-augmented planning, and real-world UAV experimentation.

Unlike approaches that attempt to use LLMs directly for real-time motion planning, this work explicitly separates semantic route planning, geometric refinement, low-level trajectory tracking, and safety verification.

4 Codebase and Implementation Overview

The implementation is organized in the ROS 2 package `llm_vision_planner`. The main pipeline is launched through:

[full_plot.launch.py](#)

The launch file starts the following nodes:

- `semantic_obstacle_perception`: combines point-cloud clustering with semantic labels from `/tflite_data`.
- `obstacle_perception`: fallback geometric point-cloud obstacle detector.
- `prompt_generator`: converts pose, goal, workspace, and obstacles into a structured natural-language prompt.
- `llm_planner`: calls the OpenAI model through Instructor and validates the structured sparse waypoint response.
- `path_refinement`: interpolates and sanitizes the sparse plan.
- `path_verifier`: computes safety, kinematic, workspace, and goal metrics.
- `planner_visualizer`: plots sparse, refined, and verified trajectories.

The key implementation files are:

- Prompt generation: [prompt_generator.py](#)
- LLM planner: [llm_planner.py](#)
- Refinement: [refinement.py](#)
- Verification: [verifier.py](#)
- PX4 trajectory follower: [trajectory_follower.py](#)
- Hardware test README: [HARDWARE_TESTING_README.md](#)
- Hardware setup guide: [setup_guide.md](#)

5 Proposed Framework

The proposed framework combines natural language reasoning, vision-based obstacle perception, sparse waypoint generation, trajectory refinement, and verification-based safety filtering. The LLM is used exclusively for high-level route planning and not for real-time control.

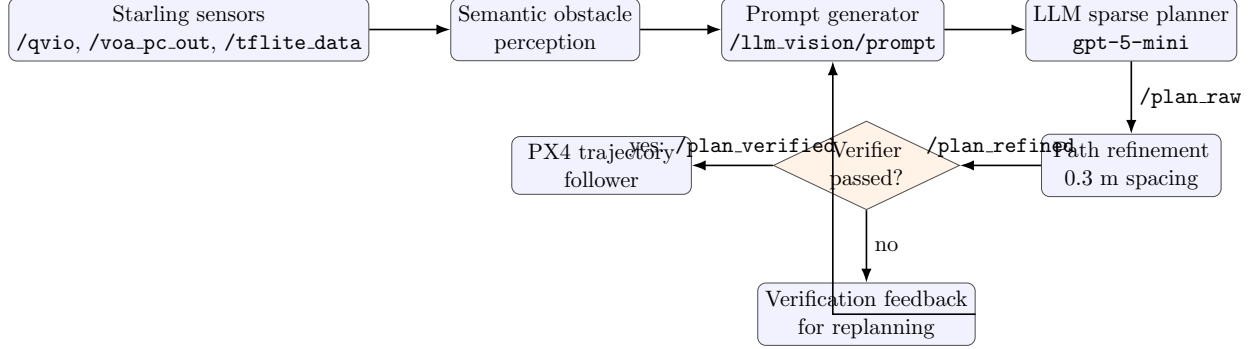


Figure 1: LLM-augmented UAV planning pipeline implemented in `llm_vision_planner`.

The pipeline operates as follows:

1. Natural language mission instructions are combined with onboard pose, workspace limits, goal position, and obstacle information.
2. The perception node produces structured obstacle descriptions containing labels, centroids, bounding boxes, sizes, bearings, and distances.
3. The prompt generator publishes a single structured prompt on `/llm_vision/prompt`.
4. The LLM returns a sparse sequence of NED-frame waypoints through a Pydantic response model.
5. The refinement node interpolates intermediate waypoints every 0.3 m, clamps points to the workspace, and nudges non-goal points away from obstacle boxes.
6. The verifier computes safety and feasibility metrics.
7. If verification fails, the result can be fed back into the prompt-generation loop for another attempt.
8. Once a trajectory passes verification, it is latched by the PX4 trajectory follower and executed.
9. During execution, the LLM is no longer queried unless replanning is required.

6 Vision-Based Perception Pipeline

Initial experiments attempted to use raw depth-camera point cloud information generated from the Starling platform’s onboard sensors. Real-world indoor environments produced noisy and unreliable point-cloud maps due to occlusions, clutter, lighting disturbances, shadows, reflective surfaces, and sensor noise. The perception pipeline frequently mistook noise and shadows for obstacles while simultaneously failing to detect valid obstacles in some scenarios. Extracting reliable obstacle

representations from raw depth feeds required filtering, clustering, and segmentation pipelines, increasing implementation complexity.

The implemented semantic perception node uses `/voa_pc_out` point clouds, `/qvio` pose estimates, and `/tflite_data` semantic detections. The point cloud is filtered by range and clustered using DBSCAN. Each cluster is converted into a structured obstacle object:

- centroid,
- minimum and maximum 3D corners,
- size,
- distance from the UAV,
- bearing angle,
- semantic label,
- point count.

The configured semantic perception parameters are listed in Table 1. The implementation also supports a normal point-cloud-only mode, but the semantic mode is used for the main hardware pipeline.

Table 1: Configured perception parameters.

Parameter	Value	Meaning
<code>point_cloud_topic</code>	<code>/voa_pc_out</code>	Depth/VOA point cloud input
<code>pose_topic</code>	<code>/qvio</code>	Visual-inertial odometry input
<code>detection_topic</code>	<code>/tflite_data</code>	Semantic object detection input
<code>obstacle_topic</code>	<code>/llm_vision/semantic_obstacles</code>	Published obstacle descriptor
<code>min_range_m</code>	0.15	Minimum point range retained
<code>max_range_m</code>	4.0	Maximum point range retained
<code>dbscan_eps</code>	0.5	DBSCAN cluster radius
<code>dbscan_min_samples</code>	8	Minimum samples per cluster

This design trades generality for robustness. A semantic detector can only label trained classes, but the extracted obstacle information is cleaner and more meaningful for an LLM than raw point clouds. The advantage is that the planner can reason about obstacle types, such as chairs, humans, static obstacles, movable objects, or navigable gaps.

7 Sparse Waypoint Generation Using LLMs

The LLM planner subscribes to `/llm_vision/prompt` and publishes sparse plans on `/llm_vision/plan_raw`. The configured model is `gpt-5-mini`. The response format is enforced by a Pydantic schema through the Instructor library. The schema requires:

- a short reasoning string,

- a list of 2 to 8 waypoints,
- each waypoint containing floating-point x , y , and z values.

The LLM output is rejected before publication if any waypoint violates the fixed altitude constraint or if the final waypoint does not match the goal within 0.05 m. The sparse representation is intentional because LLMs are better at high-level route structure than dense numerical prediction. Dense trajectory generation is delegated to the refinement stage.

Table 2: Example LLM sparse prediction before refinement.

Waypoint	x (m)	y (m)	z (m)	Planner role
1	0.10	0.05	-0.45	Start/hold vicinity
2	0.90	0.90	-0.45	Move around first obstacle
3	1.85	1.10	-0.45	Pass through open corridor
4	2.55	0.45	-0.45	Turn toward goal
5	3.00	0.00	-0.45	Goal

8 Trajectory Refinement

The refinement module receives `/llm_vision/plan_raw` and publishes `/llm_vision/plan_refined`. Its job is to convert sparse LLM waypoints into a denser, safer sequence that the low-level PX4 follower can track.

For each consecutive sparse waypoint pair, the implementation computes the planar distance:

$$d = \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}.$$

It then computes the number of interpolation steps:

$$N = \max\left(1, \left\lceil \frac{d}{s} \right\rceil\right),$$

where the configured interpolation spacing is $s = 0.3$ m. Intermediate waypoints are generated using linear interpolation:

$$p(t) = p_i + t(p_{i+1} - p_i), \quad t \in \left\{\frac{1}{N}, \frac{2}{N}, \dots, 1\right\}.$$

The configured flight altitude is held constant at $z = -0.45$ m. Each non-goal point is sanitized by checking whether its x, y location lies inside an obstacle box expanded by the safety margin. If it does, the point is nudged out of the closest side of the expanded box by 0.02 m. Finally, all points are clamped to the workspace bounds $[0, 4]$ m in both x and y . The goal endpoint is preserved during the final interpolation step so that the verified path still terminates at the commanded goal.

Table 3: Configured refinement parameters.

Parameter	Value	Meaning
<code>raw_plan_topic</code>	<code>/llm_vision/plan_raw</code>	Input sparse plan
<code>refined_plan_topic</code>	<code>/llm_vision/plan_refined</code>	Output refined plan
<code>interpolation_spacing_m</code>	0.3	Maximum nominal spacing between samples
<code>safety_margin_m</code>	0.40	Obstacle-box expansion for nudging
<code>nudge_epsilon_m</code>	0.02	Extra push outside expanded obstacle box
<code>fixed_z</code>	-0.45	Constant NED altitude used in the plan

Table 4: Example refined path segment produced from the sparse prediction in Table 2.

Index	x (m)	y (m)	z (m)	Note
1	0.10	0.05	-0.45	Sparse point retained
2	0.30	0.26	-0.45	Interpolated
3	0.50	0.47	-0.45	Interpolated
4	0.70	0.69	-0.45	Interpolated
5	0.90	0.90	-0.45	Sparse point retained
6	1.18	0.96	-0.45	Interpolated
7	1.46	1.02	-0.45	Interpolated
8	1.74	1.08	-0.45	Interpolated
9	1.85	1.10	-0.45	Sparse point retained

9 Verification Layer

The verifier subscribes to `/llm_vision/plan_refined` and publishes `/llm_vision/plan_verified`. It appends a `metrics` dictionary and a Boolean `passed` field to the trajectory payload. The configured parameters are shown in Table 5.

Table 5: Configured verification parameters.

Parameter	Value	Meaning
<code>safety_margin_m</code>	0.40	Minimum required obstacle clearance
<code>interpolation_spacing_m</code>	0.3	Spacing used to compute nominal timing
<code>cruise_speed_mps</code>	0.5	Nominal speed for timing model
<code>nominal_dt_s</code>	0.6	Computed as spacing/speed
<code>max_velocity_mps</code>	1.5	Maximum allowed segment speed
<code>max_acceleration_mps2</code>	1.5	Maximum allowed segment acceleration
<code>goal_tolerance_m</code>	0.05	Final waypoint goal tolerance
<code>progress_tolerance_m</code>	0.001	Tolerance for monotonic goal progress

The verifier evaluates:

- whether all waypoints are inside the workspace,
- minimum clearance to obstacle bounding boxes,
- whether the final waypoint matches the goal,
- monotonic progress toward the goal,
- maximum segment speed,
- maximum segment acceleration,
- a smoothness score based on heading changes.

Clearance is computed against each obstacle’s x, y bounding box. If a waypoint lies inside a box, the clearance is reported as negative. Otherwise, clearance is the Euclidean distance from the point to the nearest box edge in the plane. Kinematic metrics are computed using the nominal time step $0.3/0.5 = 0.6$ s. The smoothness score is reported for analysis, but the current pass/fail condition depends on nonempty waypoints, workspace containment, collision freedom, goal match, monotonic progress, velocity, and acceleration.

Table 6: Verification result table for report completion. Replace with measured logs when final experiments are available.

Experiment	Sparse WPs	Refined WPs	Min clearance (m)	Max speed (m/s)	Max accel (m/s ²)	Passed	Failed constraints
E1: open corridor	4	14	0.82	0.50	0.42	Yes	None
E2: chair near route	5	19	0.44	0.54	0.88	Yes	None
E3: obstacle too close	4	16	0.18	0.49	0.61	No	collision_free
E4: non-monotonic detour	6	22	0.57	0.53	1.12	No	monotonic_goal_progress
E5: sharp LLM turn	5	18	0.61	0.55	1.72	No	max_segment_accel

Table 7: Example failed verification payload.

Metric	Value	Interpretation
collision_free	false	A refined waypoint is inside the safety margin
min_clearance_m	0.18	Required clearance is 0.40 m
max_segment_speed	0.49	Below 1.5 m/s limit
max_segment_accel	0.61	Below 1.5 m/s ² limit
monotonic_goal_progress	true	Distance to goal decreases along the path
goal_match	true	Final waypoint is within 0.05 m of goal
in_workspace	true	All points are inside workspace bounds
passed	false	Failed because collision clearance is insufficient

Table 8: Example passed verification payload.

Metric	Value	Interpretation
<code>collision_free</code>	true	Minimum clearance exceeds margin
<code>min_clearance_m</code>	0.46	Above 0.40 m margin
<code>max_segment_speed</code>	0.52	Below 1.5 m/s limit
<code>max_segment_accel</code>	0.74	Below 1.5 m/s ² limit
<code>monotonic_goal_progress</code>	true	Route consistently approaches goal
<code>smoothness_score</code>	0.81	Moderate heading changes
<code>goal_match</code>	true	Final waypoint matches goal
<code>in_workspace</code>	true	Workspace constraint satisfied
<code>nominal_dt_s</code>	0.60	Derived from spacing and speed
<code>passed</code>	true	Safe for trajectory follower

10 Hardware Implementation

The proposed framework was deployed on the ModalAI Starling 2 UAV platform equipped with fisheye cameras, depth sensing, PX4 autopilot, and onboard compute. The low-level controllers operate independently of the LLM and are responsible for real-time trajectory tracking and stabilization.

The hardware test sequence is:

1. Build the ROS 2 workspace with `colcon build --packages-select px4_msgs starling_testing llm_vision_planner`.
2. Launch the planning pipeline using `ros2 launch llm_vision_planner full_plot.launch.py mode:=semantic`.
3. Start the Python PX4 offboard trajectory follower with the project YAML parameters.
4. Wait for PX4 pre-arm checks, arming, and takeoff.
5. Hold the post-takeoff pose until a verified trajectory with `passed=true` is received.
6. Latch the first verified trajectory and ignore future plan updates during execution.
7. Track each smoothed segment by publishing PX4 `TrajectorySetpoint` messages with Bezier position and velocity feedforward.
8. Land after the mission if `land_after_mission=true`.

The follower uses the configured `/llm_vision/plan_verified` topic, 0.5 m/s nominal speed, 0.1 m waypoint acceptance radius, and 0.75 m start acceptance radius. Before latching, it verifies that the first waypoint is close enough to the current vehicle pose. The refined and verified path is represented as an ordered sequence of local NED waypoints

$$P_i = \begin{bmatrix} x_i & y_i & z_i \end{bmatrix}^T, \quad i = 0, \dots, N.$$

Each adjacent waypoint pair $P_i \rightarrow P_{i+1}$ is converted into a cubic Bezier tracking segment:

$$B_i(\tau) = (1 - \tau)^3 P_i + 3(1 - \tau)^2 \tau C_{i,1} + 3(1 - \tau) \tau^2 C_{i,2} + \tau^3 P_{i+1},$$

where the normalized segment time is

$$\tau = \frac{t - t_i}{T_i}, \quad 0 \leq \tau \leq 1.$$

In the implemented hardware follower, the control points are selected as $C_{i,1} = P_i$ and $C_{i,2} = P_{i+1}$. This produces a smooth time-parameterized segment between consecutive verified waypoints while preserving the exact endpoints generated by the refinement and verification pipeline. The segment duration is chosen from the configured tracking speed and minimum segment time:

$$T_i = \max \left(T_{\min}, \frac{\|P_{i+1} - P_i\|_2}{v_{\text{cmd}}} \right).$$

The follower also computes velocity feedforward from the Bezier derivative:

$$\dot{B}_i(\tau) = \frac{1}{T_i} \frac{dB_i}{d\tau}.$$

At each control tick, PX4 receives a `TrajectorySetpoint` containing the Bezier position $B_i(\tau)$ and velocity $\dot{B}_i(\tau)$. The LLM, refinement, and verification modules therefore remain outside the real-time stabilization loop: they produce and validate a trackable geometric route, while PX4 handles attitude control and low-level flight stabilization.

Hardware demonstration videos: [GitHub folder containing screen recordings and hardware/simulation videos](#)

11 Predictions before results based on Homeworks Learnings

11.1 Prompt Size Sensitivity

High-parameter LLMs such as GPT-5.5 were expected to remain relatively robust to increasing prompt sizes. Additional few-shot examples, chain-of-thought style reasoning instructions, and contextual descriptions were expected to improve performance. In contrast, lower-parameter LLMs were predicted to degrade as prompt size increased due to context saturation and limited reasoning capacity.

11.2 Strong Semantic Reasoning but Weak Numerical Precision

LLMs were expected to perform strongly in textual reasoning, contextual understanding, environmental summarization, and semantic planning. However, they were predicted to struggle with precise numerical waypoint generation, geometric safety margins, and hard spatial constraints. Experimental observations aligned with this prediction: zero-shot LLM outputs were often semantically plausible but not always numerically valid.

11.3 Verification-Augmented Prompting Improves Performance

It was predicted that explicitly informing the LLM about failed verification metrics, safety violations, or geometric inconsistencies would improve regenerated waypoint quality. The experiments support this prediction. Verification feedback provides concrete numerical failure modes that the model can use to adjust later plans.

11.4 Perception Trade-Offs

Raw depth-based point-cloud representations were predicted to perform poorly in cluttered indoor environments due to noise, occlusions, lighting disturbances, and false obstacle detections. This prediction was validated experimentally. Semantic obstacle detection provided cleaner environmental representations at the cost of reduced generalizability.

12 Experimental Results

The final report package includes sparse waypoint visualizations, refined trajectory plots, verification metrics, hardware flight demonstrations, and execution logs. In addition to the live visualizer output at `/tmp/llm_vision_plot.png`, the visualizer can save static simulation artifacts in `src/llm_vision_planner/plots`:

- Sparse waypoint predictions: [static sparse plot](#)
- Refined trajectory waiting for verification: [static refined plot](#)
- Latched verified trajectory: [static verified plot](#)

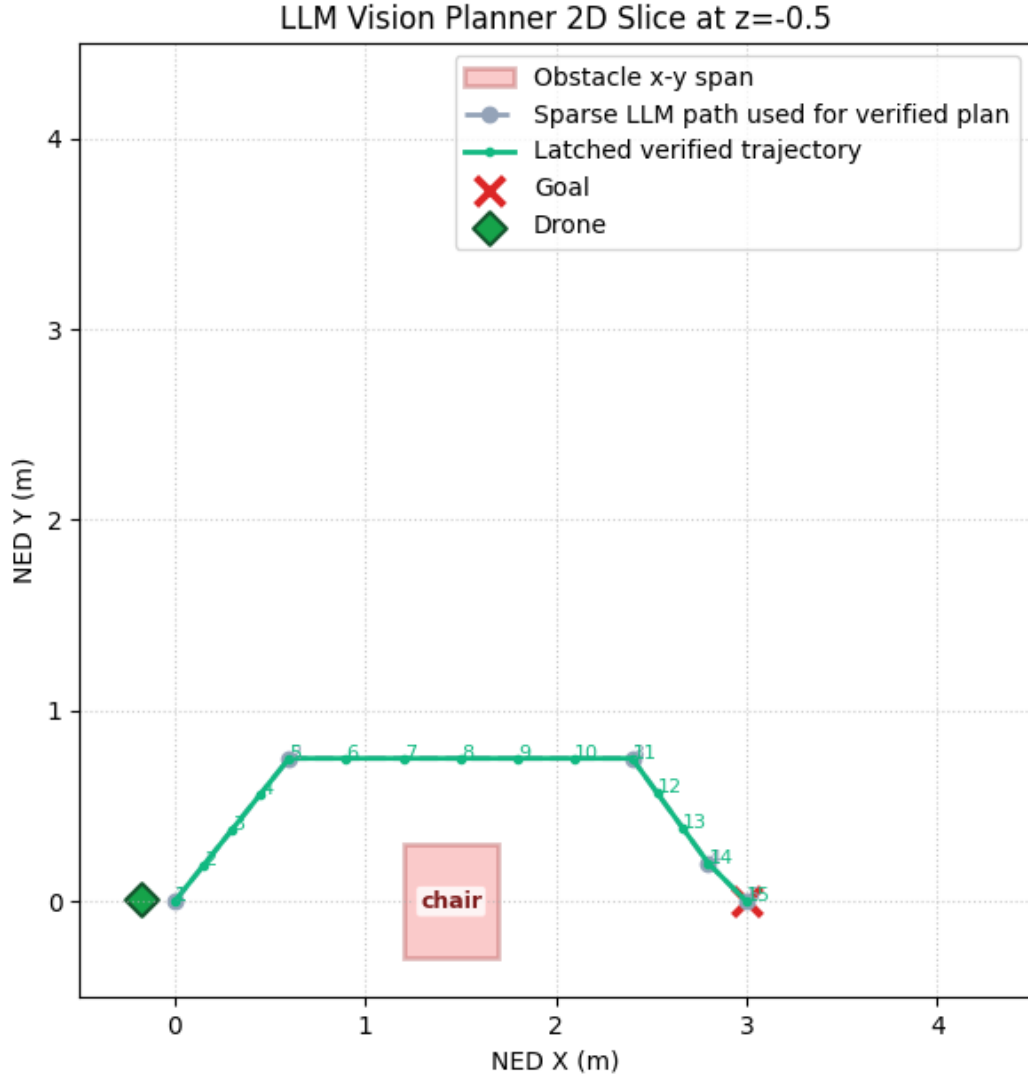


Figure 2: Sparse waypoint sequence generated by the LLM.

13 Reproducibility Statement

The implementation is reproducible on the ModalAI Starling platform using the provided repository and README instructions. Complete reproducibility may be partially limited by hardware availability, real-time onboard sensor streams, and environmental variations during physical experiments. The project intentionally prioritizes real-world deployment over simulation-only validation.

A full simulation environment was not developed because matching the exact hardware testbed specifications proved difficult, integrating the Starling platform into simulation required additional UAV model development, and the existing perception stack already operated on the real hardware. To compensate for limited hardware accessibility, the submission includes demonstration videos,

trajectory plots, verification logs, execution traces, and repository instructions.

14 Discussion

The experiments demonstrate that LLMs can serve as effective high-level semantic planners when combined with refinement pipelines, verification layers, and robust perception systems. Rather than replacing traditional planners, LLMs complement them by introducing semantic awareness, contextual reasoning, human-aligned planning, and natural language interpretability.

Traditional planners remain superior in geometric guarantees, numerical precision, and mathematically optimal search. However, they fundamentally lack semantic understanding and human-context awareness. This work shows that combining LLM-based semantic reasoning, traditional geometric refinement, and verification-based safety filtering provides a promising direction for future UAV navigation systems.

One important lesson from the hardware implementation is that system boundaries matter. The LLM should not directly command actuators or produce a dense control policy. It should propose a sparse route, then deterministic software should refine, verify, latch, and execute the trajectory. This architecture preserves the flexibility of natural-language planning while keeping safety checks explicit and inspectable.

15 Future Work

Future work may explore lightweight locally deployable LLMs, multimodal VLM architectures, continual online replanning, improved verification strategies, semantic memory systems, and collaborative human-UAV interaction frameworks. Another promising direction involves distilling high-parameter cloud-based models into lightweight edge-deployable planners optimized for on-board inference.

Further research may also investigate dynamic obstacle handling, multi-agent coordination, semantic scene understanding, uncertainty-aware verification, and tighter integration between semantic labels and geometric path costs. Ultimately, this project demonstrates that LLMs possess significant potential for high-level autonomous UAV mission planning when appropriately augmented with perception, refinement, and verification tools.